

Lab 3 - Architecture Selection and Justification

Digital Campus Library Reservation Gateway | Problem Statement #08

Name: Aarya Tedla

SRN: PES1UG24CS496

Architecture Selection

We chose a Layered Architecture for the Digital Campus Library Reservation Gateway. The system can be divided into three clear parts: a Presentation Layer for the Student Member and Head Librarian screens, a Business Layer for the actual library rules, and a Data Layer for storing catalogue, hold, member, notification and fine records. For a campus library application, this keeps the design simple enough to maintain while still separating the main responsibilities properly.

Reason 1 - Clear separation of responsibilities

The features in this system are related, but they should not all be handled in one module. Catalogue search, the FIFO hold queue, availability notifications and overdue fine processing each have their own rules. Keeping these rules in the Business Layer means the same logic can be used regardless of whether the request comes from the student portal or the librarian dashboard. It also makes future changes easier. For example, the hold-limit rule or the fine calculation can be changed without redesigning the user interface or the database structure.

Reason 2 - A practical fit for the size of the system

A microservices architecture would give independent deployment and scaling, but it would also introduce extra service communication, deployment effort and data-consistency issues. That is more complexity than this campus system needs for the lab scenario. A basic client-server design is simpler, but it tends to place too much application logic on one central server. Layered architecture gives a better middle ground: the modules are separated logically, but they can still be developed and deployed as one manageable application.

Security Advantage

The database is not accessed directly from either user-facing screen. Requests first go through the Business Layer, where login checks, role checks and input validation can be applied. This is important because a Student Member should be able to search books, place holds and view their own fine information, while actions such as managing catalogue records or changing fine-related data should be restricted to the Head Librarian. Keeping these checks in one layer reduces the chance of bypassing the rules through the interface.

Performance Benefit

Catalogue search is expected to be one of the most frequent operations, so the Catalog Service can handle pagination, indexing and caching before querying the database. The same structure keeps queue operations inside the Hold Queue Manager instead of mixing them with the user interface. Since communication between these modules stays inside the application, the design avoids the repeated network calls that a distributed microservices setup could add. This helps keep search and reservation actions responsive under normal campus usage.

How the Layers Work Together

A normal request starts at one of the two user interfaces, moves to the relevant business component, and only then accesses stored data. For example, a book search goes from the Student Member Portal to the Catalog Service and then to the database. A hold request is handled by the Hold Queue Manager, which can ask the Notification Service to alert the next member when a copy becomes available. This keeps the flow easy to follow and prevents direct interface-to-database access.